Lab 6.1: Building an ActiveX Control Using an Existing Class

In this lab, you will create an ActiveX control from an existing device-independent bitmap (DIB) class, set properties for the control, and add a property page for the control.

To see a demonstration of the solution to this lab, click this icon.
 (CD-ROM plays the demonstration, "Lab 6.1 Demonstration.")

Estimated time to complete this lab: **120 minutes**

To complete the exercises in this lab, you must have the required software. For detailed information about the labs and setup for the labs, see Labs in this course.

The code that forms the starting point for this lab is located in the folder \Labs\Ch6\Lab6.1\Ex01.

The solution code for this lab is located in the folder \\Labs\Ch6\Lab6.1.

Objectives

After completing this lab, you will be able to:

- Package an existing class as an ActiveX control.
- Use the ActiveX Control Test Container.
- Add property pages to an ActiveX control.

Prerequisites

Before working on this lab, you should be familiar with the following:

- Using the ControlWizard to create an ActiveX control
- Setting ActiveX control properties
- Implementing property pages for ActiveX controls

Exercises

The following exercises provide practice working with the concepts and techniques covered in this chapter.

Exercise 1: Building an ActiveX Control

In this exercise, you will create an ActiveX control by encapsulating the existing device-independent bitmap (DIB) class. After you create the control, you will test the control using the ActiveX Control Test Container.

Exercise 2: Setting Properties for an ActiveX Control

In this exercise, you will set properties for the ActiveX control you created in Exercise 1.

Exercise 3: Adding a Property Sheet Interface

In this exercise, you will use a property sheet to create a design mode interface for the ActiveX control you implemented in Exercise 2.

193 Exercise 1: Building an ActiveX Control

The code that forms the basis for this exercise is in \Labs\Ch06\Lab6.1\Ex01. After you have created your project, copy the files into your project folder.

In this exercise, you will create an ActiveX control. You will use an existing device-independent bitmap (DIB) class. After you create the control, you will test the control by using the ActiveX Control Test Container.

▶ Build the framework control

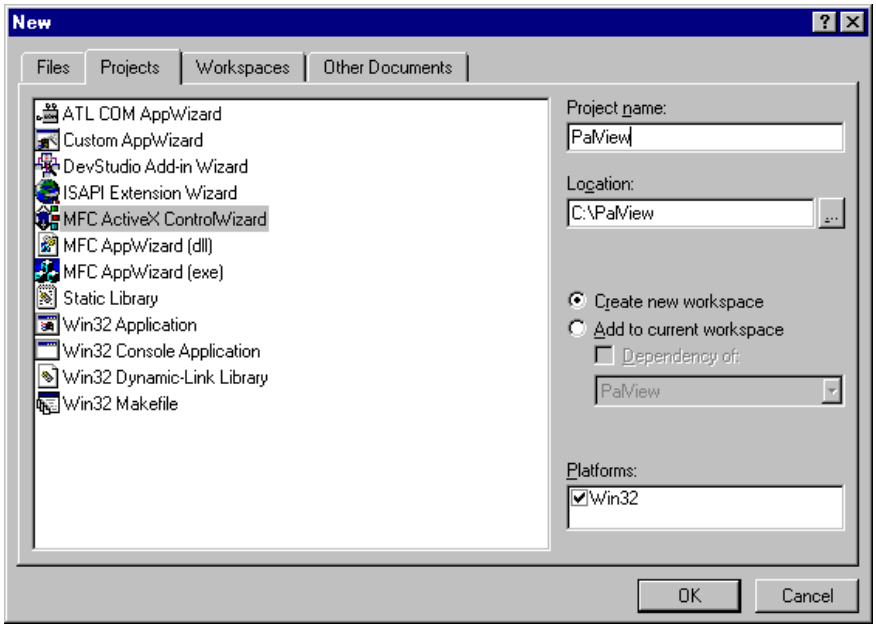
☐ 1. Start Microsoft Visual Studio 6.0.

☐ 2. On the **File** menu, click **New**.

The **New** dialog box appears.

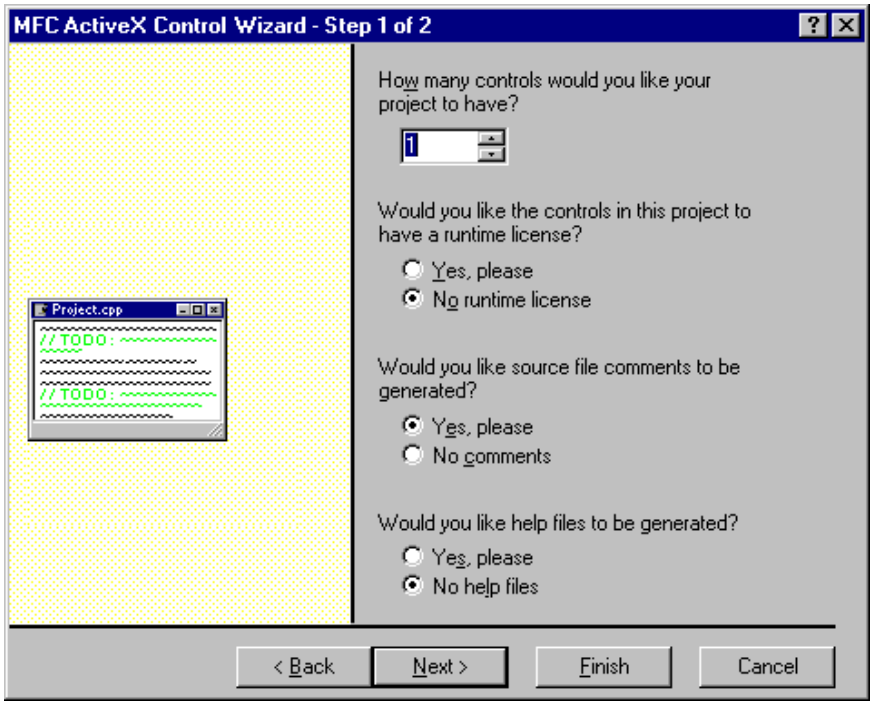
☐ 3. On the **Projects** tab, click **MFC ActiveX ControlWizard**, set the project name to PalView and click **OK** to start ControlWizard. To see an illustration of the **New** dialog box, click this icon.

(CD-ROM displays the following illustration.)



Notes

☐ 4. In Step 1 of ControlWizard, create one control, leave the other options at their default settings, and click **Next**. To see an illustration of **MFC ActiveX ControlWizard - Step 1** dialog box, click this icon. (CD-ROM displays the following illustration.)

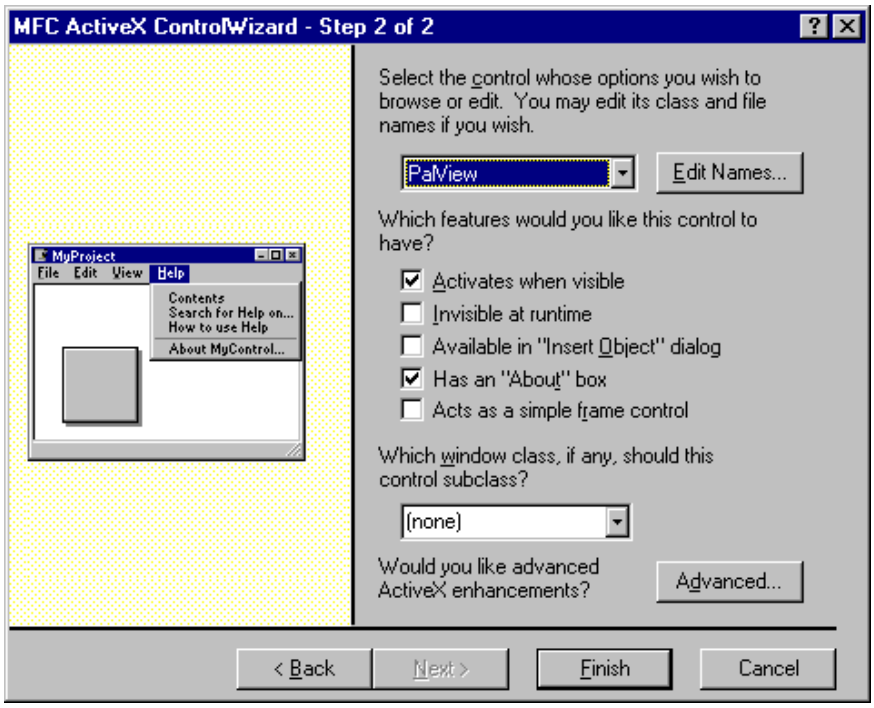


Notes

☐ 5. In Step 2 of ControlWizard, accept the default settings for all options and click **Finish**.

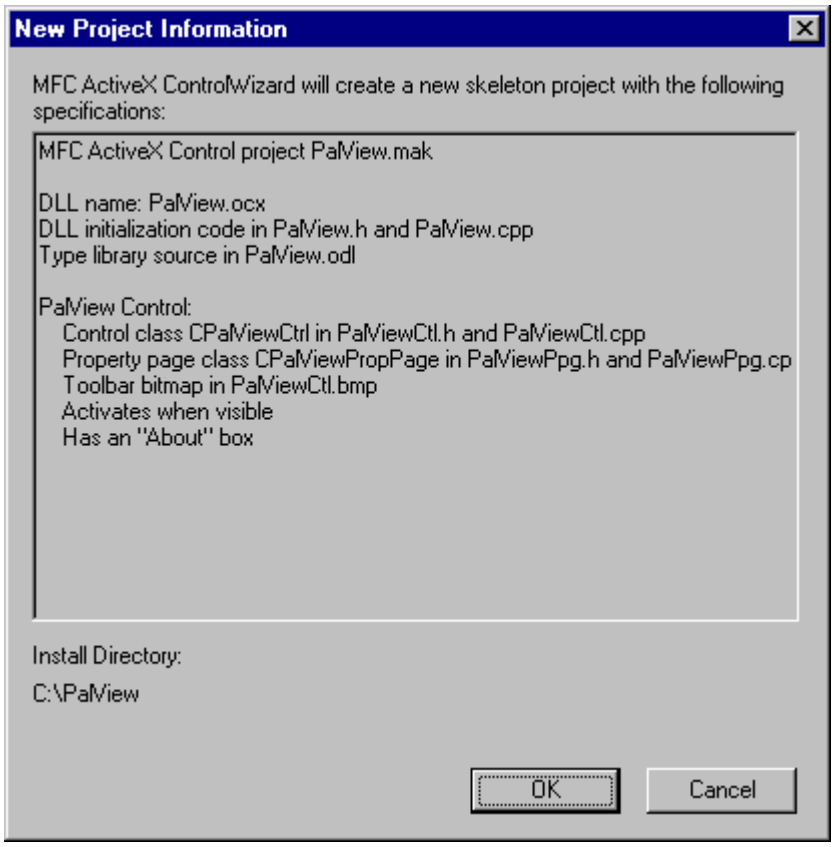
The **New Project Information** dialog box appears.

To see an illustration of **MFC ActiveX ControlWizard - Step 2** dialog box, click this icon. (CD-ROM displays the following illustration.)



Notes

To see an illustration of the **New Project Information** dialog box, click this icon. (CD-ROM displays the following illustration.)



☐ 6. Click **OK** to create the project.

☐ 7. On the **Project** menu, click **Add to Project**, and click **Files**.

Insert the files Dib.cpp and DibPal.cpp into the project. You do not need to insert their headers files; Visual Studio will resolve their dependencies.

The **CDIB** class creates a device-independent bitmap from an existing bitmap file, and the **CDIBPal** class uses this information to determine the colors used to create the bitmap.

▶ **Integrate CDIBPal into the CPalViewCtrl control**

☐ 1. Right-click **CPalViewCtrl** in ClassView and add a protected member variable. The protected member variable will be defined as follows:

```
CDIBPal*    m_pDibPal;
```

This pointer will be used to refer to the **CDIBPal** object.

☐ 2. Open the file PalViewCtl.cpp.

☐ 3. Include the files Dib.h and DibPal.h before the file PalViewCtl.h.

Notes

☐ 4. Update the constructor to initialize the m_pDibPal message to **NULL**. To see an example of how your code should look, click this icon.


(CD-ROM displays the following sample code.)

```
CPalViewCtrl::CPalViewCtrl()
{
    InitializeIIDs(&IID_DPalView, &IID_DPalViewEvents);
    m_pDibPal = NULL;
}
```

☐ 5. Update the destructor to delete m_pDibPal if it exists. To see an example of how your code should look, click this icon.


(CD-ROM displays the following sample code.)

```
CPalViewCtrl::~CPalViewCtrl()
{
    if (NULL != m_pDibPal)
        delete m_pDibPal;
}
```

▶ **Update the CPalViewCtrl::OnDraw function to use the CDibPal::Draw function**

☐ 1. Modify **CPalViewCtrl::OnDraw** to use **CDIBPal::OnDraw** if **CDIBPal** has been instantiated, as follows:

```
if(NULL != m_pDibPal)
    m_pDibPal->Draw(pdc, rcBounds, TRUE);
```

Otherwise, fill the control with a white brush as shown in the following code:

```
else
    pdc->FillRect(rcBounds,
        CBrush::FromHandle((HBRUSH)GetStockObject(WHITE_BRUSH)));
```

☐ 2. Save PalViewCtl.cpp. To see an example of how your code should look, click this icon.


(CD-ROM displays the following sample code.)

```
void CPalViewCtrl::OnDraw(
CDC* pdc, const CRect& rcBounds, const CRect& rcInvalid)
{
    if(NULL != m_pDibPal)
        m_pDibPal->Draw(pdc, rcBounds, TRUE);
    else
        pdc->FillRect(rcBounds,
            CBrush::FromHandle((HBRUSH)GetStockObject(WHITE_BRUSH)));
}
```

Notes

▶ **Add a function to extract the palette from a DIB file**

☐ 1. Right-click **CPalViewCtl** in ClassView and add a protected member function as follows:

```
void ExtractPalFromDIBFile(LPCTSTR lpszFileSpec)
```

The function will be used to extract the color palette from the DIB file.

☐ 2. Add the following local variables to the implementation of the **ExtractPalFromDIBFile**:

```
CDIB        Dib; //use to load Device Independent Bitmap
CFile   DibFile; //used to open bitmap file
```

☐ 3. Open the bitmap file, using the path stored in lpszFileSpec as follows:

```
if(DibFile.Open(lpszFileSpec, CFile::modeRead))
```

☐ 4. Copy the file into a **CDIB** instance as follows:

```
if(Dib.Load(&DibFile))
```

☐ 5. Create a palette and extract the color table from the DIB into the palette as follows:

```
CDIBPal* pTemp = new CDIBPal;
ASSERT(pTemp != NULL);
if(pTemp->Create(&Dib))
```

☐ 6. Add the following code to delete the instance of **CDIBPal**, if it exists.

```
if(NULL != m_pDibPal)
{
    delete m_pDibPal;
    m_pDibPal = NULL;
}
```

☐ 7. Assign the palette you created to m_pDibPal as follows:

```
m_pDibPal = pTemp;
```

☐ 8. Draw the palette in 3-D style as follows:

```
m_pDibPal->SetDraw3D(TRUE);
```

Notes

☐ 9. Save PalViewCtl.cpp. To see an example of how your code should look, click this icon.

 (CD-ROM displays the following sample code.)

```
void CPalViewCtrl::ExtractPalFromDIBFile(LPCSTR lpszFileSpec)
{
    CDIB        Dib;
    CFile   DibFile;

    if(DibFile.Open(lpszFileSpec, CFile::modeRead))
    {
        // Load the bitmap file into a CDIB object
        if(Dib.Load(&DibFile))
        {
            CDIBPal* pTemp = new CDIBPal;
            ASSERT(pTemp != NULL);
            //  This will extract the color table fron the CDIB object
            if(pTemp->Create(&Dib))
            {
                // Get rid of any existing CDIBPal object
                if(NULL != m_pDibPal)
                {
                    delete m_pDibPal;
                    m_pDibPal = NULL;
                }
                // Ready to go
                m_pDibPal = pTemp;

                // Make sure the new palette reflects our
                // current state
                m_pDibPal->SetDraw3D(TRUE);
            }
            else
            {
                delete pTemp;
            }
        }
    }
}
```

Notes

(CD-ROM displays the following tip.)

The Microsoft Foundation Class Library provides two mechanisms for implementing a property:

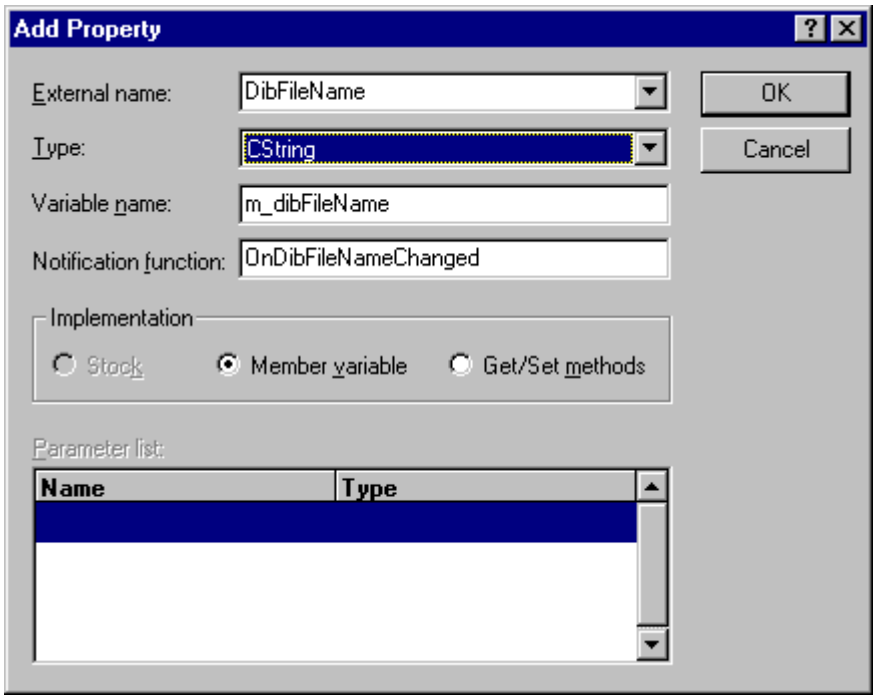
- Member variables with notification
- **Get** and **Set** methods

In general member variables are used when the property is a simple stored value. Use **Get** and **Set** methods for those properties that update the user interface, or require other calculations to take place when their value changes.

▶ Add DibFileName property

☐ 1. In ClassWizard, click the **Automation** tab and click **Add Property**. The **Add Property** dialog box appears. To see an illustration of the **Add Property** dialog box, click this icon.

(CD-ROM displays the following illustration.)



☐ 2. With **CPalViewCtrl** as the class, create a new **DibFileName** property, using the following settings.

External name	Type	Variable name	Notification function
DibFileName	**CString**	m_dibFileName	**OnDibFileNameChanged**

Notes

☐ 3. In PalViewCtl.cpp, modify the new **OnDibFileNameChanged** function to extract the palette from **DibFileName** and repaint the control. To see an example of how your code should look, click this icon.

(CD-ROM displays the following sample code.)

```
void CPalViewCtrl::OnDibFileNameChanged()
{
    ExtractPalFromDIBFile(m_dibFileName);
    InvalidateControl();
    SetModifiedFlag();
}
```

☐ 4. In the constructor, initialize m_dibFileName as follows:

```
m_dibFileName = _T ("");
```

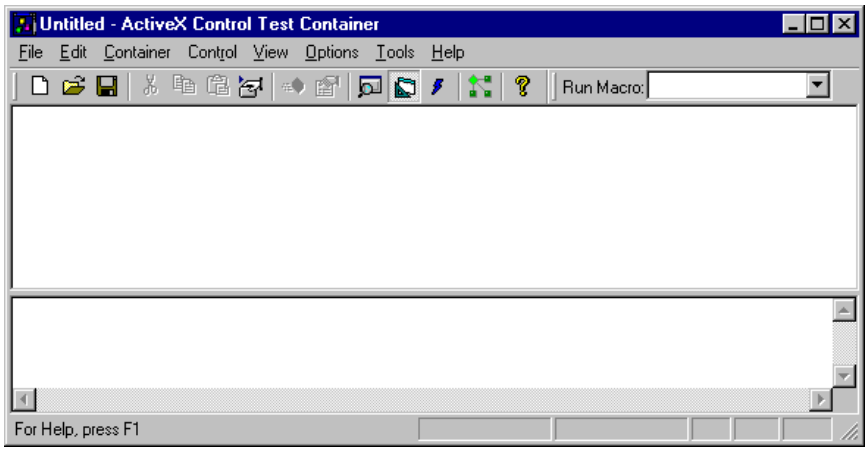
▶ **Build and run the control using the ActiveX Control Test Container**

☐ 1. Save all files and build the file PalView.ocx.

☐ 2. Microsoft Visual Studio provides a test container for ActiveX controls. To run the test container, click **ActiveX Control Test Container** on the **Tools** menu.

To see an illustration of the ActiveX Control Test Container, click this icon.

(CD-ROM displays the following illustration.)



☐ 3. On the **Edit** menu, click **Insert New Control**, then click **PalView**. Resize the control to an appropriate size.

☐ 4. On the **Edit** menu, click **Properties**.

The **Properties** dialog box appears. Because you have not implemented a property sheet for this control, invoking **Properties** displays only the blank default property sheet provided by ControlWizard.

The completed code for this exercise is in \Labs\Ch06\Lab6.1\Ex01\Solution.

Notes

193 Exercise 2: Setting Properties for an ActiveX Control

Continue with the files you created in Exercise 1 or, if you do not have a starting point for this exercise, the code that forms the basis for this exercise is in \Labs\Ch06\Lab6.1\Ex02.

In this exercise, you will set the properties for the ActiveX control you created in Exercise 1.

▶ Implement the ShowColorSelection and Show3D properties

☐ 1. The **ShowColorSelection** and **Show3D** properties are member variables of the **CPalViewCtrl** class. They are implemented in the same way as the **DibFileName** property in Exercise 1, as member variables with notification functions. Using ClassWizard, create the following properties.

External name	Type	Variable name	Notification function
ShowColorSelection	**BOOL**	m_showColorSelection	**OnShowColorSelectionChanged**
Show3D	**BOOL**	m_show3D	**OnShow3DChanged**

☐ 2. When **ShowColorSelection** is changed, redraw the control to show or hide the red outlining rectangle indicating the color tile selected. Invalidate the control in **OnShowColorSelectionChanged**:

```
void CPalViewCtrl::OnShowColorSelectionChanged()
{
    InvalidateControl();
    SetModifiedFlag();
}
```

☐ 3. Call **SetDraw3D** in **OnShow3DChanged**. Pass m_show3D as a parameter to **SetDraw3D**.

```
void CPalViewCtrl::OnShow3DChanged()
{
    if (NULL != m_pDibPal)
    {
        m_pDibPal->SetDraw3D(m_show3D);
        InvalidateControl();
    }
    SetModifiedFlag();
}
```

▶ Implement the SelectionIndex property

☐ 1. In **CPalViewCtrl**, implement the **SelectionIndex** property as a pair of **Get**/**Set** methods as follows:

External name	Type	Get	Set
SelectionIndex	**short**	**GetSelectionIndex**	**SetSelectionIndex**

Notes

☐ 2. To get the selection index, use **CDIBPal::GetSelectionIndex** to query the control. Return –1 if you do not have a **CDIBPal** instance:

```
short CPalViewCtrl::GetSelectionIndex()
{
    short nSelectionIndex = -1;
    if (NULL != m_pDibPal)
    {
        nSelectionIndex = m_pDibPal->GetSelectionIndex();
    }
    return nSelectionIndex;
}
```

☐ 3. To set the selection index, use **CDIBPal::SetSelectionIndex**:

```
void CPalViewCtrl::SetSelectionIndex(short nNewValue)
{
    if(nNewValue >= 0 && nNewValue < 256)
    {
        if (NULL != m_pDibPal)
        {
            m_pDibPal->SetSelectionIndex(nNewValue);
            FireSelChange(nNewValue);
            InvalidateControl();
        }
    }
    SetModifiedFlag();
}
```

In the previous example, the **FireSelectionChange** function fires the selection change event, which has not yet been added to the ActiveX control being created.

Notes

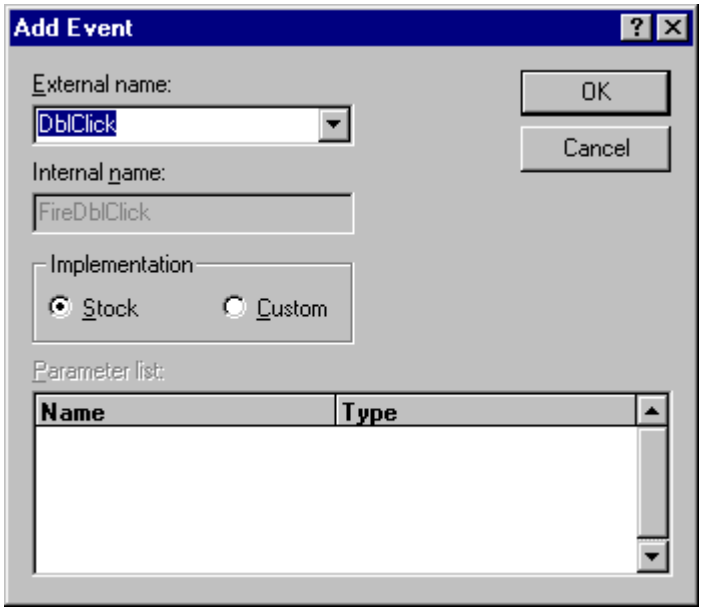
▶ **Add event handling for left-click and double-click**

☐ 1. Start ClassWizard.

☐ 2. On the **ActiveX Events** tab, click **Add Event**.

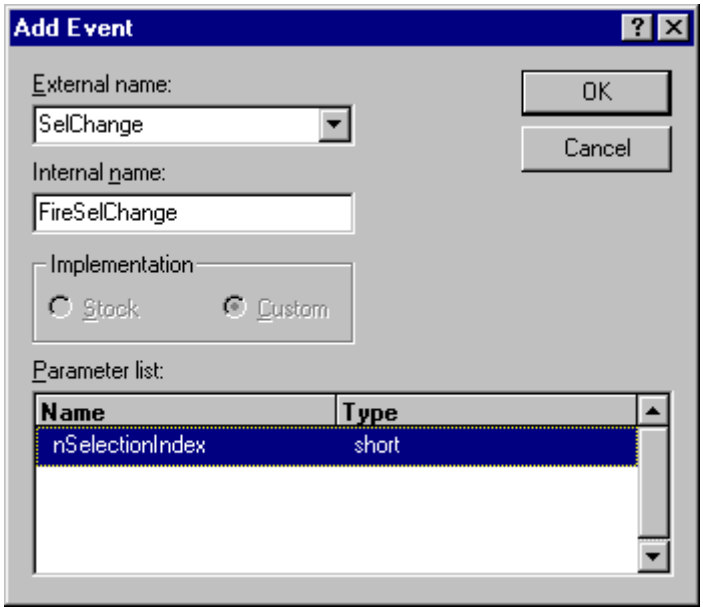
The **Add Event** dialog box appears.

☐ 3. Add a stock event for **DblClick**, as shown in the following illustration.



Notes

☐ 4. Add a custom event for **SelChange**. Add a short parameter, nSelectionIndex.



☐ 5. Open the **Message Maps** tab in ClassWizard and add a handler for the WM_LBUTTONDOWN message in the **CPalViewCtrl** class.

☐ 6. In the code for **OnLButtonDown**, validate that the ActiveX control has instantiated a **CDIBPal** object, and if so, get the size of the control:

```
if(NULL != m_pDibPal)
{
    int x, y;
    GetControlSize(&x, &y);
    CRect rcControl(0, 0, x, y);
```

☐ 7. Pass **CRect** and **CPoint** to **CDIBPal::HitTest** to get the cell that was clicked. Then, set the selection index:

```
int nPalIndex = m_pDibPal->HitTest(rcControl, point);
if(-1 != nPalIndex)
{
    SetSelectionIndex(nPalIndex);
}
```

Notes

☐ 8. Save PalViewCtl.cpp. To see code for the completed function, click this icon.

 (CD-ROM displays the following sample code.)

```
void CPalViewCtrl::OnLButtonDown(UINT nFlags, CPoint point)
{
    if(NULL != m_pDibPal)
    {
        int x, y;
        GetControlSize(&x, &y);
        CRect rcControl(0, 0, x, y);

        //  Ask the DibPal object which palette index would
        //  be hit by the mouse click
        int nPalIndex = m_pDibPal->HitTest(rcControl, point);
        if(-1 != nPalIndex)
            //Change the selection property
            SetSelectionIndex(nPalIndex);
    }
    COleControl::OnLButtonDown(nFlags, point);
}
```

▶ **Implement a method to get the current color from the palette**

☐ 1. On the **Automation** tab in ClassWizard, add a method to get a color from the palette.

External name	Return type	Parameter name	Parameter type
GetColorFromPalette	**long**	nPalIndex	**short**

☐ 2. Edit the code for the **GetColorFromPalette** method as follows:

a. Check that the palette index is between 0 and 255:

```
COLORREF        crRet = 0; // Default to BLACK
if(nPalIndex >= 0 && nPalIndex < 256)
{
```

b. Return CDIBPal::GetColorFromIndex if there is a CDIBPal object:

```
if(NULL != m_pDibPal)
{
    crRet = m_pDibPal->GetColorFromIndex(nPalIndex);
}
return crRet;
}
```

Notes

☐ 3. Save PalViewCtl.cpp. To see code for the complete function, click this icon.

(CD-ROM displays the following sample code.)

```
long CPalViewCtrl::GetColorFromPalette(short nPalIndex)
{
    COLORREF        crRet = 0; // Default to BLACK
    if(nPalIndex >= 0 && nPalIndex < 256)
    {
        if(NULL != m_pDibPal)
            crRet = m_pDibPal->GetColorFromIndex(nPalIndex);
    }
    return crRet;
}
```

▶ **Modify the functions provided by ControlWizard**

DoPropExchange is called by the framework when loading or storing a control from a persistent storage representation, such as a stream or property set. This function normally makes calls to the PX_ family of functions to load or store specific user-defined properties of an ActiveX control. Add PX_ calls for the properties you just implemented as follows:

```
PX_Bool(pPX, _T("ShowColorSelection"), m_showColorSelection, TRUE);
PX_Bool(pPX, _T("Show3D"), m_show3D, FALSE);
PX_String(pPX, _T("DibFileName"), m_dibFileName, _T(""));
```

☐ 1. After loading DibFileName, get the palette from that file:

```
if (pPX->IsLoading())
{
    ExtractPalFromDIBFile(m_dibFileName);
}
```

☐ 2. Save PalViewCtl.cpp. To see code for the complete function, click this icon.

(CD-ROM displays the following sample code.)

```
void CPalViewCtrl::DoPropExchange(CPropExchange* pPX)
{
    ExchangeVersion(pPX, MAKELONG(_wVerMinor, _wVerMajor));
    COleControl::DoPropExchange(pPX);

    PX_Bool(pPX, _T(“ShowColorSelection”), m_showColorSelection, TRUE);
    PX_Bool(pPX, _T(“Show3D”), m_show3D, FALSE);
    PX_String(pPX, _T(“DibFileName”), m_dibFileName, _T(“”));

    if (pPX->IsLoading())
    {
        ExtractPalFromDIBFile(m_dibFileName);
    }

}
```

Notes

☐ 3. In the **CPalViewCtrl** constructor, initialize m_showColorSelection to **TRUE** and m_show3D to **FALSE**. To see code for the complete constructor, click this icon.

 (CD-ROM displays the following sample code.)

```
CPalViewCtrl::CPalViewCtrl()
{
    InitializeIIDs(&IID_DPalView, &IID_DPalViewEvents);
    m_pDibPal = NULL;
    m_dibFileName = _T("");

    m_showColorSelection = TRUE;
    m_show3D = FALSE;


}
```

☐ 4. In **ExtractPalFromDIBFile**, modify the call to **CDIBPal::SetDraw3D** to use m_show3D:

```
m_pDibPal->SetDraw3D(m_show3D);
```

☐ 5. In **CPalViewCtrl::OnDraw**, modify the call to **CDIBPal::Draw** to use m_showColorSelection:

```
m_pDibPal->Draw(pdc, rcBounds, m_showColorSelection);
```

☐ 6. Save PalViewCtl.cpp. Build and run PalView.ocx using the ActiveX Control Test Container.

The completed code for this exercise is in \Labs\Ch06\Lab6.1\Ex02.

Notes

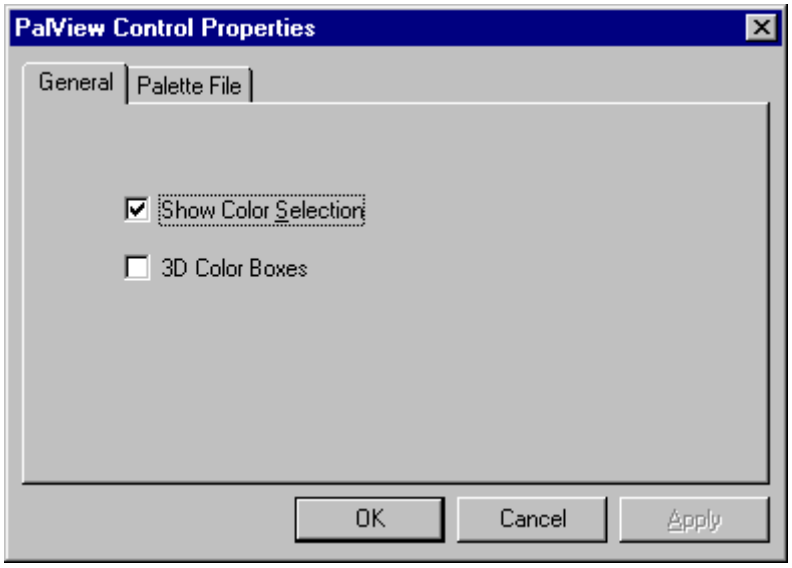
193 Exercise 3: Adding a Property Sheet Interface

Continue with the files you created in Exercise 2 or, if you do not have a starting point for this exercise, the code that forms the basis for this exercise is in \Labs\Ch06\Lab6.1\Ex03.

In this exercise, you will use a property sheet to create a design mode interface for the ActiveX control you implemented in Exercise 2. ControlWizard provides a property sheet with one blank page.

▶ Modify the property page provided by MFC AppWizard

The first property page will have two check boxes: one for the **ShowColorSelection** property and the other for the **Show3D** property as shown in the following illustration.



☐ 1. Delete the static text control on the IDD_PROPPAGE_PALVIEW dialog box template. Add and position controls on this template according to the following table.

Type	ID	Caption
Check box	IDC_COLORSELECTION	Show Color &Selection
Check box	IDC_3D_BOXES	&3D Color Boxes

☐ 2. Create member variables and link them to their properties according to the following table. Press the CTRL key, and double-click **controls** to display their variables.

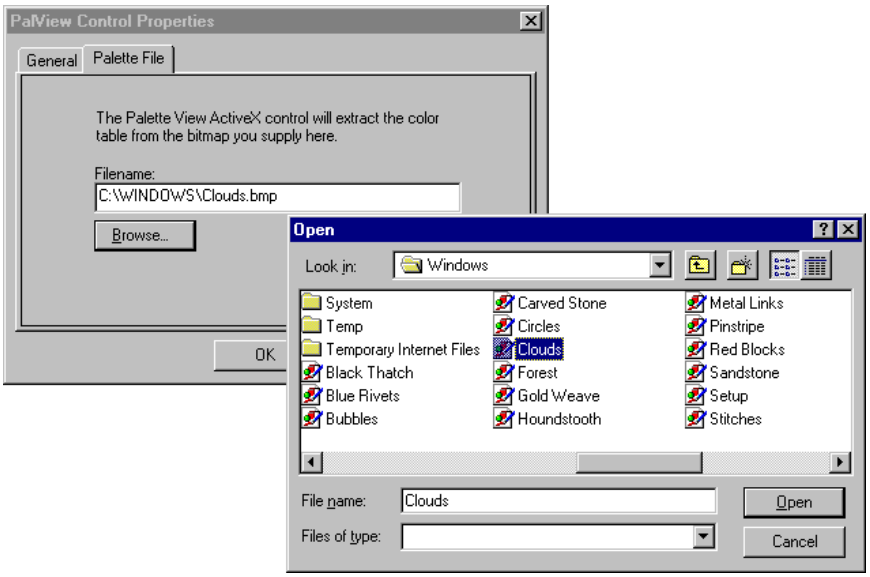
ID	Type	Variable name	Property name
IDC_COLORSELECTION	**BOOL**	m_bShowColorSelection	**ShowColorSelection**
ICD_3D_BOXES	**BOOL**	m_bShow3D	**Show3D**

Notes

▶ Add a new property page

The new property page will enable your users to set the palette to be displayed. To see an illustration of a sample palette property page, click this icon.

 (CD-ROM displays the following illustration.)



☐ 1. Create a new dialog template. On the **Insert** menu, click **Resource** to display the properties resources. Expand the Dialog icon, click **IDD_OLE_PROPPAGE_SMALL**, and add this dialog box template with the following controls.

Type	ID	Caption
Dialog	IDD_PROPPAGE_DIBFILE	None
PushButton	IDC_BROWSE	&Browse
Edit control	IDC_DIB_FILENAME	None
Static	IDC_STATIC	Filename:
Static	IDC_STATIC	The Palette View ActiveX control will extract the color table from the bitmap file you supply here.

☐ 2. Use ClassWizard to create a new class based on the IDD_PROPPAGE_DIBFILE dialog box. Make the base class **COlePropertyPage**. Name the new class **CDibFilePropPage** and use DibPrpg.cpp and DibPrpg.h as file names.

☐ 3. Create a member variable for the edit control.

ID	Type	Variable name	Property name
IDC_DIB_FILENAME	**CString**	m_DibFileName	**DibFileName**

Notes

☐ 4. In **CDibFilePropPage**, create a command handler for the click event of the IDC_BROWSE message.

☐ 5. In the **CDibFilePropPage::OnBrowse** function, display a common file dialog box and set the text of the file name edit control with the results. To see code for the complete function, click this icon.


(CD-ROM displays the following sample code.)

```
void CDibFilePropPage::OnBrowse()
{
    CFileDialog dlg(TRUE);
    if(dlg.DoModal() == IDOK)
    {
        CEdit * pEdit = (CEdit *)GetDlgItem(IDC_DIB_FILENAME);
        if(0 != pEdit)
            pEdit->SetWindowText(dlg.GetPathName());
    }
}
```

☐ 6. In the string table resource, add two strings for captions.

ID	Caption
IDS_DIBPAGE	Palette File
IDS_DIBPAGE_CAPTION	Palette File

☐ 7. In the **UpdateRegistry** function of DibPrpg.cpp, modify the call to **AfxOleRegisterPropertyPageClass** to use IDS_DIBPAGE instead of 0. Note that ClassView does not display this member function, which is located in the .cpp file. To see an example of how your code should look, click this icon.


(CD-ROM displays the following sample code.)

```
BOOL CDibFilePropPage::CDibFilePropPageFactory::UpdateRegistry(BOOL
bRegister)
{
    // TODO: Define string resource for page type;
    // replace '0' below with ID.
    if (bRegister)
        return AfxOleRegisterPropertyPageClass(AfxGetInstanceHandle(),
            m_clsid, IDS_DIBPAGE);
    else
        return AfxOleUnregisterClass(m_clsid, NULL);
}
```

Notes

☐ 8. In the **CDibFilePropPage::CDibFilePropPage** function, change the initialization to use IDS_DIBPAGE_CAPTION instead of 0. To see code for the complete function, click this icon.

 (CD-ROM displays the following sample code.)

```
CDibFilePropPage::CDibFilePropPage() :
    COlePropertyPage(IDD, IDS_DIBPAGE_CAPTION)
{
    //{{AFX_DATA_INIT(CDibFilePropPage)
    m_DibFileName = _T("");
    //}}AFX_DATA_INIT
}
```

☐ 9. Save this file.

▶ Setup the new property page in CPalViewCtrl

☐ 1. Include DibPrpg.H in PalViewCtl.cpp.

☐ 2. Change the BEGIN_PROPPAGEIDS macro to indicate that you will have two pages in the property sheet as follows:

```
BEGIN_PROPPAGEIDS(CPalViewCtrl, 2)
```

☐ 3. Add an additional PROPPAGEID macro for **CDibFilePropPage** as follows:

```
PROPPAGEID(CDibFilePropPage::guid)
```

☐ 4. Save PalViewCtl.cpp. The following code shows the complete macro block:

```
BEGIN_PROPPAGEIDS(CPalViewCtrl, 2)
    PROPPAGEID(CPalViewPropPage::guid)
    PROPPAGEID(CDibFilePropPage::guid)
END_PROPPAGEIDS(CPalViewCtrl)
```

☐ 5. Build and use the control.

The completed code for this exercise is in \Labs\Ch06\Lab6.1\Ex03\Solution.

Notes